Github Agentic Workflow

https://github.github.com/gh-aw/setup/quick-start/

https://github.github.com/gh-aw/introduction/how-they-work/

https://github.com/github/gh-aw/tree/v0.45.5/.github/workflows

What are agentic workflows?

Traditional GitHub Actions workflows execute pre-programmed steps with fixed if/then logic. They do exactly what you tell them, every time, in the same way. Agentic workflows take a fundamentally different approach. You write natural language instructions in Markdown, and an AI coding agent interprets them and uses your repository's context to make decisions and take appropriate actions.

Like other workflow files, these workflows live in your .github/workflows/ directory. It has two parts. The top section is YAML frontmatter that configures when the workflow runs, what it can access, and what it's allowed to do. The bottom section is Markdown that describes the task in natural language. Here's an example that creates a daily status report for repository maintainers in the file .github/workflows/daily-repo-status.md:

Once the file is ready, run gh aw compile, which transforms the .md file into a .lock.yml file – a security-hardened GitHub Actions workflow with schema validation, action SHA pinning, and other protections baked in. Next, you commit both files, and the workflow runs automatically on your defined schedule

You will need a fine-grained Personal Access Token (PAT) from your user account that has Copilot Requests permissions. This must be stored as GITHUB_COPILOT_TOKEN in your repository secrets. The workflow will not run without this token. That permission is only available when the resource is your user account and the access is either "All repositories" or "Public repositories". Essentially, it associates your account for Copilot billing purposes and provides a way for the workflows to call Copilot.

How they execute

Under the hood, the system supports multiple AI engines. GitHub Copilot is the default, but you can also configure Claude by Anthropic or OpenAI Codex. The engine interacts with your repository through tools exposed via the Model Context Protocol (MCP). Each tool the agent can access is explicitly declared in the frontmatter, so you always know what capabilities you're granting. You can also use Safe Inputs – special scripts that run in isolation and have controlled access to secrets – to safely extend the agent's capabilities with custom tools.

Your Markdown is converted into an Actions workflow using gh aw (an open source extension written in Go). The workflow uses a series of Actions steps and jobs to implement the steps, running the agent tasks in sandboxes with strict security boundaries. It uses the provided token to allow those runs to access Copilot, and the GitHub Token (by default) to access the repository and tools.

When a workflow triggers, the execution follows a carefully staged pipeline designed to keep the agent's capabilities tightly controlled:

1. **Pre-activation**

The system checks role permissions and validates that the lock file matches the source Markdown.

2. **Input sanitization**

Incoming event content – issue bodies, PR descriptions, comment text – is cleaned before the agent sees it. This includes neutralizing @mentions, protecting against bot loops, converting HTML to safe formats, filtering URLs to HTTPS-only from trusted domains, and enforcing content size limits.

3. **Agent execution**

The agent runs with read-only permissions. It can read your repository, issues, and pull requests through MCP tools and safe inputs, but it cannot directly write anything. Any actions it wants to take – like creating an issue or opening a pull request – are buffered as structured artifacts rather than executed immediately. This helps prevent the agent from taking any direct write actions if it were compromised or hallucinating.

4. **Threat detection**

A separate AI-powered analysis job downloads those buffered artifacts and examines them for secret leaks, malicious code patterns, and policy violations. This detection agent runs in complete isolation from the original agent, with its own security-focused prompt. That reduces the risk of one of the inputs compromising the prompt.

5. **Safe output execution**

Only after threat detection passes do separate, scoped jobs execute the requested write operations – each with the minimum permissions required for that specific action. This isolates the buffered artifacts from the code that updates the repository. This provides deterministic updates without giving the agent direct write access and provides a clear audit trail. By default, there is a special safe output called missing-data that encourages agents to explicitly report when they lack the data needed to complete a task, rather than hallucinating an answer. This

rewards honest AI behavior and creates a natural feedback loop for improving your agentic workflows.

Here's the critical insight: the AI agent gets broad read access to your repository, issues, and PRs. It can analyze anything. But it can only perform write operations through safe-outputs you explicitly define in the frontmatter. Want the agent to add a comment? Declare add-comment: max: 1. Want it to create a PR? Declare create-pull-request. If it's not declared, it can't happen.

The security model

The security architecture is built on the premise that AI agents are most dangerous when they have private data access, untrusted inputs, and external communication channels. To address this, it operates across three trust layers that work together to contain failures.

Substrate-level trust provides the foundation. The agent runs inside a containerized environment on a GitHub Actions runner, with kernel-enforced memory, CPU, and process isolation. The Agent Workflow Firewall (AWF) routes all network traffic through a Squid proxy that enforces a domain allowlist, and each MCP server runs in its own isolated container. These provide some guarantees even if a component is fully compromised and executes arbitrary code. The proxy and firewall are configurable, and they support deep packet inspection of HTTPS traffic.

Configuration-level trust constrains what gets loaded and how components connect. A trusted compiler validates workflow definitions against a strict schema, pins action references to specific SHA hashes (preventing supply chain attacks from tag hijacking), and runs security scanners including actionlint, zizmor, shellcheck, and poutine. Authentication tokens are treated as imported capabilities whose distribution is explicitly controlled through declarative configuration. You can tighten this further with gh aw compile --strict, which enforces additional constraints like prohibiting write permissions and requiring explicit network configuration.

Plan-level trust constrains behavior over time. The compiler decomposes a workflow into stages, each with defined permissions, data outputs, and rules for how subsequent stages can consume them. Artifacts are created and passed through AI-powered threat detection. After that, separate jobs with scoped write permissions execute the actual changes. The blast radius of any compromise is limited to the stage where it occurs.

```
gh extension install github/gh-aw

gh aw --version

#configure your respository
gh aw init

#if you need to login
gh auth login --hostname github.enterprise.com

gh aw add githubnext/agentics/daily-repo-status

gh aw add-wizard githubnext/agentics/daily-repo-status
```

Create a workflow for GitHub Agentic Workflows using https://raw.githubusercontent.com/github/gh-aw/main/create.md

The purpose of the workflow is a daily report on recent activity in the repository, delivered as an issue.

gh aw add-wizard https://github.com/github/gh-aw/blob/v0.45.5/.github/workflows/issue-arborist.md

Open the workflow markdown file located at .github/workflows/daily-repo-status.md

After modify the markdown file, compile it again as

gh aw compile

#run the pipeline

gh aw run daily-repo-status

#if any error is coming, then chance are that workflow is not in the main or default branch

gh aw new # Interactive mode

gh aw new my-custom-workflow # Create template (.md extension optional)

gh aw new my-workflow --force # Overwrite if exists

gh aw new my-workflow --engine claude # Inject engine into frontmatter

Auto-fix deprecated workflow fields using codemods. Runs in dry-run mode by default; use --write to apply changes.

Terminal window

gh aw fix # Check all workflows (dry-run)

gh aw fix --write # Fix all workflows

gh aw fix my-workflow --write # Fix specific workflow

gh aw fix --list-codemods # List available codemods

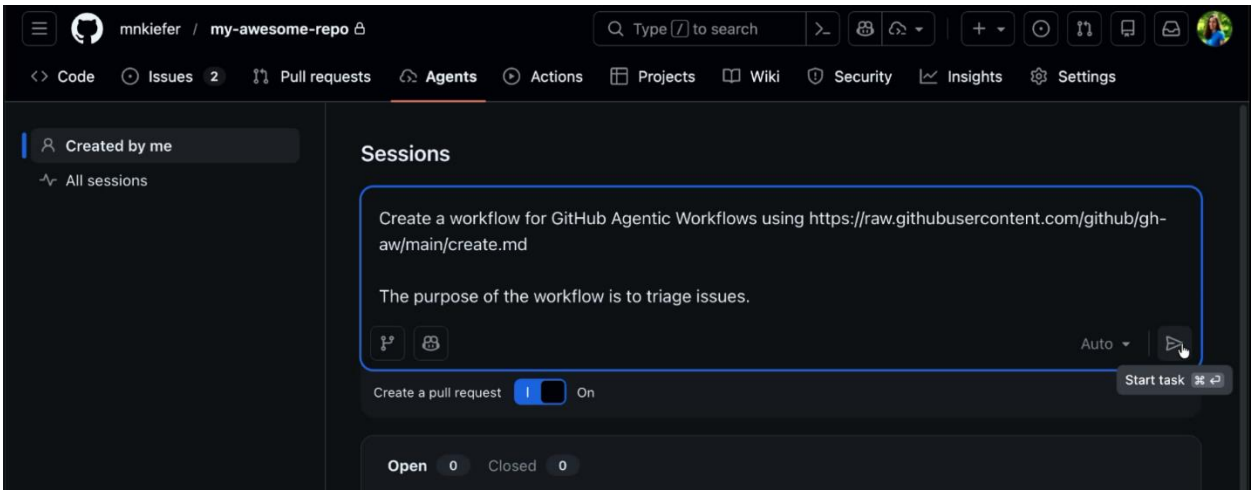
```
gh aw list                          # List all workflows
gh aw list ci-                      # Filter by pattern (case-insensitive)
gh aw list --json                   # Output in JSON format
gh aw list --label automation       # Filter by label
gh aw list --dir custom/workflows   # List from custom directory

gh aw status                        # All workflows
gh aw status --ref main             # With run info for main branch
gh aw status --label automation     # Filter by label
gh aw status --repo owner/other-repo  # Check different repository

gh aw logs workflow                 # Download logs for workflow
gh aw logs -c 10 --start-date -1w   # Filter by count and date
gh aw logs --ref main --parse --json  # With markdown/JSON output for branch

gh aw audit 12345678                # By run ID
gh aw audit https://github.com/owner/repo/actions/runs/123 # By workflow run URL
gh aw audit https://github.com/owner/repo/actions/runs/123/job/456 # By job URL (extracts first failing step)
gh aw audit https://github.com/owner/repo/actions/runs/123/job/456#step:7:1 # By step URL (extracts specific step)
gh aw audit 12345678 --parse        # Parse logs to markdown
gh aw audit 12345678 --repo owner/repo  # Specify repository for bare run ID
```

#create agentic workflow using Github UI

GitHub Web Interface

If you have access to GitHub Copilot, you can create and edit Agentic Workflows directly from the Web Interface. This technique is slow and non-interactive but it is incredibly useful to turn an idea to reality in a couple minutes. For a more fine grained, interactive experience we recommend using a coding agent (see next section).

From <https://github.github.com/gh-aw/setup/creating-workflows/>



Create a workflow for GitHub Agentic Workflows using https://raw.githubusercontent.com/github/gh-aw/main/create.md

The purpose of the workflow is to triage issues: label them by type and priority, identify duplicates, ask clarifying questions when the description is unclear, and assign them to the right team members.

Create a workflow for GitHub Agentic Workflows using https://raw.githubusercontent.com/github/gh-aw/main/create.md

Every day, check if there's a new release from astro for any of my dependencies. If yes, then look at the changelog as well as the "upgrading astro" docs. Come up with a plan for upgrading astro, including dealing with any breaking changes. And then create a PR with all the changes related to the upgrade.

- Changelog: https://github.com/withastro/astro/blob/main/packages/astro/CHANGELOG.md
- Upgrading Astro guide: https://docs.astro.build/en/upgrade-astro/